

Continuous Fourier Transform (CFT)

Step 1: The 2-Minute Intuition (What Fourier Actually Is)

Think of any signal (or image) like a musical chord played on a piano:

- **Time Domain (or Space Domain):** This is the sound wave hitting your ear (or the image pixels on your screen). It looks messy and complicated.
- **Fourier Domain:** This is the sheet music listing the exact notes (frequencies) and how loud each note is (magnitude/amplitude).

Why do properties exist?

Properties just tell you what happens to the sheet music when you mess with the song:

1. **Time Shift (Delay):** If you pause the song and play it 2 seconds later, the notes (frequencies) don't change. You just hit play later (a phase shift: multiply by $e^{-j2\pi ft_0}$).
2. **Derivative (Rate of change):** Fast-changing notes (high pitches) get louder. Slow notes (low bass) get quieter (multiply by $j2\pi f$).
3. **Scaling (Speed up):** If you play the song at $2\times$ speed, all the pitches get higher (frequency spectrum widens).

That is literally the entire concept.

Step 2: Master the "Universal 5-Line Recipe"

Every single problem they throw at you—whether it's shifting, derivatives, scaling, or modulation—follows the **exact same 5-step script**. Memorize this workflow:

```
# 1. Compute FT of original
X_orig = cft_obj.compute_cft() # (or calculate_all_coefficients)

# 2. Modify the signal in time/space (e.g. shift, differentiate)
x_modified = np.roll(x_orig, shift_amount)

# 3. Compute FT of the modified signal (NUMERICAL METHOD)
cft_mod = CFT_Class(x_modified)
X_numerical = cft_mod.compute_cft()

# 4. Compute the THEORETICAL prediction from the property table
X_theoretical = X_orig * np.exp(-1j * 2 * np.pi * f * t0) # (Formula from table)

# 5. Verify and compute MSE
mse = np.mean(np.abs(X_numerical - X_theoretical)**2)
print("MSE:", mse)
```

If you can write those 5 steps, you can solve **95% of any question** they give you.

Step 3: The "Math → Python" Translation Cheat Sheet

Keep this mapping in your head (or on your notes). When you see a math formula on the question paper, translate it to Python like this:

Math Operation / Property	Math Formula	Python Translation
Time/Spatial Shift	$X(f) \cdot e^{-j2\pi f t_0}$	<code>X_orig * np.exp(-1j * 2 * np.pi * f * t0)</code>
Derivative	$j2\pi f \cdot X(f)$	<code>1j * 2 * np.pi * f * X_orig</code>
2nd Derivative / Laplacian	$-(2\pi f)^2 \cdot X(f)$	<code>-(2 * np.pi * f)**2 * X_orig</code>
Frequency Shift (Modulation)	$X(f - f_0)$	<code>np.roll(X_orig, shift_index)</code>
Linear Scaling	$A \cdot x(t) + B$	<code>A * X_orig</code> (and add B only at $f = 0$)
Magnitude Spectrum	$\ X(f)\ $	<code>np.abs(X)</code>
Phase Spectrum	$\angle X(f)$	<code>np.angle(X)</code>
Mean Squared Error	$\frac{1}{N} \sum \ A - B\ ^2$	<code>np.mean(np.abs(A - B)**2)</code>

Step 4: Pre-Exam Setup & Verification

Spend 45–60 minutes doing these concrete tasks:

Task 1: Clean Your Base Files (15 mins)

Open your IDE right now. Make sure `fs_redrawer.py` and `cft_edge_detector.py` run from terminal with **zero errors**.

- Run: `python3 fs_redrawer.py svgs/heart.svg 150`
- Run: `python3 cft_edge_detector.py pikachu.png out.png 15`
- If they run and save the outputs without crashing, your "engine" is 100% ready.

Task 2: Create a `sandbox.py` Testbed (20 mins)

Create a new file named `testbed.py` in the same directory. Write down the helper functions:

```
import numpy as np

def calculate_mse(arr1, arr2):
    return np.mean(np.abs(arr1 - arr2)**2)

def get_coeff_array(fs_obj):
```

```
sorted_keys = sorted(fs_obj.coeffs.keys())
return np.array([fs_obj.coeffs[k] for k in sorted_keys], dtype=complex)
```

Try running **just one** verification on your SVG heart:

- Multiply the SVG signal by a phase: `z_shifted = z * np.exp(1j * np.pi / 4)`
- Verify that the new coefficients equal `c_orig * np.exp(1j * np.pi / 4)`.
- Compute the MSE. Watch it print a tiny number like `1e-16`.
- *Once you see that number, your confidence will click into place.*

Step 5: Tomorrow in the Exam Checklist

1. **Read the question carefully:** Identify which property it asks for (e.g., *Time Shift, Derivative, Modulation*).
2. Look up the property in the table.
3. Apply the 5-step recipe.
4. Calculate the MSE. If the MSE is small ($< 10^{-2}$ or near 0), you know your answer is 100% correct before you even submit.

Continuous Fourier Series (CFS)

Here is your tactical, zero-fluff preparation guide for the **Continuous Fourier Series (CFS)**, tailored specifically to your `FourierEpicycles` class and the 30-minute exam format.

Step 1: The Universal 5-Line Recipe for Fourier Series

Every single Fourier Series property question follows this exact script. Memorize this sequence:

```
# 1. Load signal & compute original coefficients
t, z = load_svg_path("svgs/heart.svg", num_points=1000)
fs_orig = FourierEpicycles(t, z, n_harmonics=150)
fs_orig.calculate_all_coefficients()
c_orig = np.array([fs_orig.coeffs[n] for n in sorted(fs_orig.coeffs.keys())],
dtype=complex)
n_arr = np.arange(-fs_orig.N, fs_orig.N + 1)

# 2. Modify the signal in time (e.g., shift, differentiate, rotate)
z_mod = np.roll(z[:-1], 25) # roll excluding duplicate end point
z_mod = np.append(z_mod, z_mod[0]) # restore closed interval

# 3. Compute Fourier Series of modified signal (NUMERICAL METHOD)
fs_mod = FourierEpicycles(t, z_mod, n_harmonics=150)
fs_mod.calculate_all_coefficients()
c_numerical = np.array([fs_mod.coeffs[n] for n in sorted(fs_mod.coeffs.keys())],
```

```
dtype=complex)

# 4. Compute THEORETICAL prediction from the property formula
t0 = t[25]
c_theoretical = c_orig * np.exp(-1j * n_arr * fs_orig.omega * t0)

# 5. Calculate MSE to verify
mse = np.mean(np.abs(c_numerical - c_theoretical)**2)
print("MSE:", mse) # Should be ~ 0
```

Step 2: The "Math → Python" Translation Cheat Sheet (Fourier Series)

In your code, `n_arr = np.arange(-N, N + 1)` and `w0 = fs.omega = 2*np.pi / fs.T`. Here is how every table property translates directly into **one line of NumPy code**:

Property / Operation	Math Relationship	Python Translation for <code>c_theoretical</code>
Linearity / Scale	$A \cdot x(t)$	<code>A * c_orig</code>
Spatial Shift (DC)	$x(t) + (x_0 + jy_0)$	<code>c_theo = c_orig.copy(); c_theo[N] += (x0 + 1j*y0)</code>
Time Shift	$x(t - t_0) \leftrightarrow c_n e^{-jn\omega_0 t_0}$	<code>c_orig * np.exp(-1j * n_arr * fs.omega * t0)</code>
Rotation (in 2D plane)	$x(t) \cdot e^{j\theta}$	<code>c_orig * np.exp(1j * theta)</code>
Time Reversal	$x(-t) \leftrightarrow c_{-n}$	<code>c_orig[::-1]</code>
Conjugation	$x^*(t) \leftrightarrow c_{-n}^*$	<code>np.conj(c_orig[::-1])</code>
Differentiation	$\frac{d}{dt}x(t) \leftrightarrow jn\omega_0 c_n$	<code>1j * n_arr * fs.omega * c_orig</code>
Integration ($c_0 = 0$)	$\int x(t)dt \leftrightarrow \frac{c_n}{jn\omega_0}$	<code>np.where(n_arr != 0, c_orig / (1j * n_arr * fs.omega), c_numerical[N])</code>
Freq Shift (Modulation)	$x(t)e^{jM\omega_0 t} \leftrightarrow c_{n-M}$	<code>c_theo = c_orig[:-M]; c_num = c_numerical[M:]</code>
Multiplication in Time	$x(t) \cdot y(t) \leftrightarrow c_n * d_n$	<code>np.convolve(c_x, c_y, mode='same')</code>
Periodic Convolution	$x(t) \otimes y(t) \leftrightarrow T \cdot c_n d_n$	<code>fs.T * c_x * c_y</code>
Parseval's Power	$\frac{1}{T} \int \ x(t)\ ^2 dt = \sum \ c_n\ ^2$	<code>np.trapezoid(np.abs(z)**2, t)/fs.T</code> vs <code>np.sum(np.abs(c_orig)**2)</code>

Step 3: The 4 Exam Traps (Avoid These!)

1. The Closed-Interval Trap ($z[0] == z[-1]$):

- `svg_utils.load_svg_path` returns an array where the first and last point are identical to close the loop.
- If you reverse or roll `z` directly (`z[::-1]`), you will duplicate the wrong endpoint and cause a spike in the derivative/integral.
- **Safe way to reverse:** `z_rev = np.append(z[-2::-1], z[0])` or reverse only `z[:-1]` and close with `z[0]`.
- **Safe way to roll:** `z_rolled = np.append(np.roll(z[:-1], shift), np.roll(z[:-1], shift)[0])`.

2. The Dictionary Indexing Trap:

- `fs.coeffs` is a dictionary with keys `-N, ..., 0, ..., N`.
- When converting to an array with `sorted()`, index `0` corresponds to harmonic $-N$, index `N` corresponds to harmonic `0` (the DC component), and index `2N` is harmonic $+N$.
- Always write `n_arr = np.arange(-fs.N, fs.N + 1)` alongside your array so your indices align.

3. The Integration $n = 0$ Division by Zero:

- If the question asks to verify the integration property, you **must** use `np.where(n_arr != 0, ...)` to skip $n = 0$, because dividing by `0` will give `NaN` and ruin your MSE calculation.

4. Time Scaling α :

- If they ask for $y(t) = x(2t)$, do **not** interpolate the `z` array.
- Just pass a compressed time array `t_new = t / 2.0` into `FourierEpicycles(t_new, z, N)`. The coefficients will be identical to `c_orig`.

Step 4: Tonight's 10-Minute Drill

Run this quick test script in your IDE right now to lock in the muscle memory:

```
import numpy as np
from svg_utils import load_svg_path
from fs_redrawer import FourierEpicycles

# 1. Setup
t, z = load_svg_path("svgs/heart.svg", num_points=1000)
fs = FourierEpicycles(t, z, n_harmonics=100)
fs.calculate_all_coefficients()

c_orig = np.array([fs.coeffs[n] for n in sorted(fs.coeffs.keys())], dtype=complex)
n_arr = np.arange(-fs.N, fs.N + 1)

# 2. Drill: Differentiation (Velocity of drawing)
dt = t[1] - t[0]
z_diff = np.gradient(z, dt)
```

```

fs_diff = FourierEpicycles(t, z_diff, n_harmonics=100)
fs_diff.calculate_all_coefficients()
c_diff_num = np.array([fs_diff.coeffs[n] for n in sorted(fs_diff.coeffs.keys())],
dtype=complex)

c_diff_theo = 1j * n_arr * fs.omega * c_orig

mse = np.mean(np.abs(c_diff_num - c_diff_theo)**2)
print("Differentiation MSE:", mse)

```

If it prints an MSE below **0.05**, you are completely ready for any Fourier Series question tomorrow.

Master Guide: Exam Traps, Subtle Bugs & Edge Cases

Here are the most brutal **exam traps, subtle bugs, and edge cases** that cost students 15–20 minutes of debugging panic during a timed test, divided by category.

Category 1: Mathematical & Phase Traps

(Why your MSE is huge when your code is "right")

Trap 1: The 2π Phase-Wrapping Explosion

- **The Bug:** You compute the phase difference $\angle Y(f) - \angle X(f)$, and your MSE is massive (e.g., 15.8 instead of 0.0).
- **Why it happens:** Phase in Python (`np.angle`) is strictly bounded in $(-\pi, \pi]$. If the true phase is 3.14 and your calculated phase is -3.14 , they represent the **exact same angle**. But $(3.14 - (-3.14))**2 \approx 39.5!$
- **The Fix (Wrap your phase differences):**

```

# WRONG (will explode on boundaries):
mse_phase = np.mean((np.angle(c_num) - np.angle(c_theo))**2)

# CORRECT (wraps the difference back into [-pi, pi]):
phase_diff = np.angle(c_num) - np.angle(c_theo)
phase_diff_wrapped = (phase_diff + np.pi) % (2 * np.pi) - np.pi
mse_phase = np.mean(phase_diff_wrapped**2)

```

Trap 2: Floating-Point "Phase Noise" on Near-Zero Coefficients

- **The Bug:** You verify the phase of an odd/even wave (or after differentiation), and the phase MSE is completely random noise.
- **Why it happens:** When a coefficient magnitude is near zero (e.g., $|c_n| \approx 10^{-17}$ due to machine precision), `np.angle(c_n)` computes $\arctan(\text{noise}/\text{noise})$, which produces completely random angles between $-\pi$ and $+\pi$.
- **The Fix (Mask out near-zero amplitudes before checking phase):**

```
# Only calculate phase error on harmonics that actually have signal energy!
significant_mask = np.abs(c_orig) > 1e-4

phase_diff = np.angle(c_num[significant_mask]) -
np.angle(c_theo[significant_mask])
phase_diff_wrapped = (phase_diff + np.pi) % (2 * np.pi) - np.pi
mse_phase = np.mean(phase_diff_wrapped**2)
```

Category 2: Fourier Series Specific Traps

Trap 3: The Negative Frequency Sign in Derivatives ($n < 0$)

- **The Bug:** Half of your reconstructed derivative wave is inverted or distorted.
- **Why it happens:** The differentiation rule is $j n \omega_0 c_n$. For negative harmonics ($n < 0$), n must be negative (e.g., -3).
- If you write `1j * abs(n) * omega * c_n` or iterate over `1..N`, negative frequencies get the wrong sign.
- **The Fix:**

```
# CORRECT: Always define a signed n_arr from -N to +N
n_arr = np.arange(-fs.N, fs.N + 1)
c_theo = 1j * n_arr * fs.omega * c_orig # Automatically handles negative signs
correctly!
```

Trap 4: The Frequency Shift Array Slicing Misalignment

- **The Bug:** When verifying $e^{jM\omega t}x(t) \leftrightarrow c_{n-M}$, `calculate_mse(c_num[M:], c_orig[:-M])` throws a shape mismatch error or compares the wrong indices.
- **Why it happens:**
- If $M > 0$ (positive shift to the right): `c_num[M:]` matches `c_orig[:-M]`.
- If $M < 0$ (negative shift to the left): `c_num[:M]` matches `c_orig[-M:]`.
- **The Fix (Direction-safe slicing helper):**

```

M = -3 # Shift left by 3
if M > 0:
    c_num_valid = c_num[M:]
    c_theo_valid = c_orig[:-M]
elif M < 0:
    c_num_valid = c_num[:M]
    c_theo_valid = c_orig[-M:]
else:
    c_num_valid = c_num
    c_theo_valid = c_orig

print("Freq Shift MSE:", calculate_mse(c_num_valid, c_theo_valid))

```

Trap 5: Forgetting the T Scaling in Periodic Convolution

- **The Bug:** Your convolution MSE is off by a constant multiplier (like 39.4 if $T = 2\pi$).
- **Why it happens:** Continuous periodic convolution is:

$$\frac{1}{T} \int_0^T x(\tau)y(t-\tau)d\tau \leftrightarrow c_n d_n \implies \int_0^T x(\tau)y(t-\tau)d\tau \leftrightarrow \mathbf{T} \cdot c_n d_n$$

- **The Fix:** If your integral is a raw integral over $[0, T]$ without the $\frac{1}{T}$ in front, you **must** multiply by `fs.T` in the frequency domain:

```
c_theo = fs.T * c_x * c_y # Do NOT forget fs.T!
```

Category 3: 2D Continuous Fourier Transform Traps

Trap 6: The "Complex Image Breaks the Skeleton" Trap

- **The Bug:** The question asks you to modulate an image: $I_{new}(x, y) = I(x, y)e^{j2\pi(u_0x+v_0y)}$. You pass this complex image to `CFT2D(img_new)`, and it crashes or outputs gibberish.
- **Why it happens:** Look closely at your assignment skeleton in `cft_edge_detector.py`. The `compute_cft` method was explicitly written for **real-valued images**:

```
integrand = self.I * np.cos(...) # Assumes self.I is real!
```

If `self.I` is complex, multiplying it by `cos` and `sin` mixes up the real and imaginary parts.

- **The Fix (Split complex images into Real and Imaginary parts):**


```
# When your image becomes complex (e.g. after modulation):
img_complex_arr = img.image * np.exp(1j * 2 * np.pi * (u0 * X + v0 * Y))

# Step 1: Create two ContinuousImage copies
img_r = copy.deepcopy(img)
img_r.image = img_complex_arr.real

img_i = copy.deepcopy(img)
img_i.image = img_complex_arr.imag

# Step 2: Compute CFT separately using Linearity:  $F\{R + jI\} = F\{R\} + j F\{I\}$ 
Re_r, Im_r = CFT2D(img_r).compute_cft()
Re_i, Im_i = CFT2D(img_i).compute_cft()

F_total = (Re_r + 1j * Im_r) + 1j * (Re_i + 1j * Im_i)
```

Trap 7: The 2D Axis Transposition (**axis=0** vs **axis=1**)

- **The Bug:** Taking a partial derivative $\frac{\partial I}{\partial x}$ and multiplying by $j2\pi U$ produces a high MSE.
- **Why it happens in NumPy arrays:**
- **Rows** = y -axis (vertical) $\rightarrow$ **axis=0**, conjugate to frequency v / array **V**.
- **Columns** = x -axis (horizontal) $\rightarrow$ **axis=1**, conjugate to frequency u / array **U**.
- **The Fix:**

```
dx = img.x[1] - img.x[0]
dy = img.y[1] - img.y[0]
U, V = np.meshgrid(cft.u, cft.v) # U corresponds to x (cols), V to y (rows)

# Partial derivative w.r.t x: axis=1, multiply by U
I_x = np.gradient(img.image, dx, axis=1)
F_theo_x = 1j * 2 * np.pi * U * F_orig

# Partial derivative w.r.t y: axis=0, multiply by V
I_y = np.gradient(img.image, dy, axis=0)
F_theo_y = 1j * 2 * np.pi * V * F_orig
```

Trap 8: Modifying Image Objects Without **copy.deepcopy**

- **The Bug:** You modify an image for Part (b), and suddenly your Part (a) verification starts failing.
- **Why it happens:** In Python, **img2 = img** does not create a copy; it creates a reference. If you do **img2.image = modified_array**, you just destroyed **img.image** in the original object too!

- **The Fix:**

```
import copy
img_new = copy.deepcopy(img) # Deep copies the object and all internal NumPy
arrays
img_new.image = modified_array
```

Emergency Debugging Checklist (If your test MSE is > 1.0)

Before panicking, check these 5 things in order:

1. Did you forget 2π ?

- FS exponent: $-1j * n * fs.omega * t$ (where $\omega = 2\pi/T$ is already built in).
- CFT exponent: $-1j * 2 * np.pi * u * x$ (here you **must** write $2 * np.pi$).

2. Did you put a minus sign where a plus sign belongs?

- Time delay $x(t - t_0) \rightarrow e^{-j\cdots}$ (minus in exponent).
- Time advance $x(t + t_0) \rightarrow e^{+j\cdots}$ (plus in exponent).

3. Did you forget j in derivatives?

- 1st Derivative: $1j * 2 * np.pi * f * X$
- 2nd Derivative: $(1j * 2 * np.pi * f)**2 * X = -(2 * np.pi * f)**2 * X$ (Notice the negative sign, because $j^2 = -1$!).

4. Did you do 2D roll with the wrong axis?

- Shifting in x (horizontal) is `np.roll(arr, shift_x, axis=1)`.
- Shifting in y (vertical) is `np.roll(arr, shift_y, axis=0)`.

5. Are your arrays aligned?

- Always check array shapes before calculating MSE:

```
print(arr1.shape, arr2.shape)
```